

Memory Leak — Խորը ուղեցույց

C++ հարցազրույցի ամբողջական presentation

Այս ուղեցույցը մանրամասն բացատրում է memory leak-ը՝ ինչ է, ինչու է, ինչպես հայտնաբերել, ինչպես չսխալվել, ինչպես օգտագործել tools-եր, և ինչպես հարցազրույցի վրա պատրաստվել:

1. Ի՞նչ է Memory Leak-ը

Memory leak-ը ինչ է, ինչու է, ինչպես հայտնաբերել, ինչպես չսխալվել, ինչպես օգտագործել tools-եր, և ինչպես հարցազրույցի վրա պատրաստվել:

```
void leak() {  
    int* p = new int(10);    // heap-um 4 byte  
    // delete p;    <- ՄՈՐԱԿԱՆՔ  
}
```

// p օչնական է (stack), ինչպես heap-ի memory-ն MNUM է

p-ն (stack pointer) օչնական է function-ի դրսև, ինչպես new-ով վերականգնվում է heap memory-ն ինչպես azatvel, և ինչպես չսխալվել ինչպես pointer ինչպես cuyc talis dran -> **leak**.

2. Ինչու է վտանգավոր

- Program-ի memory-ն անհատական է entacqum
- Verjum` out of memory (bad_alloc), crash
- Erkarazhamket program-ner (server, embedded) -> aveli vtangavor
- Performance ankum (swapping)

Kartkuk ("one-time") program-um mek leak-y kritik che (OS-y azatum e verjum), ինչպես server/loop-um kutakvum e u sqanum program-y.

3. Uđenataracvac patcharnery (real orinakner)

3.1 delete mořacum

```
int* p = new int[100];  
// ... ogtagorcum  
// delete[] p; <- mořacan
```

3.2 Early return / exception

```
void f() {  
    int* p = new int(5);  
    if (someError) return; // LEAK -- delete chi kanchvum  
    delete p;  
}
```

3.3 Pointer verabashxum (overwrite)

```
int* p = new int(1);  
p = new int(2); // araji new int(1)-y kordza -> LEAK  
delete p; // miayn erkrordy azatum
```

3.4 delete vs delete[] sxal

```
int* arr = new int[100];  
delete arr; // SXAL -- petq e delete[] -> undefined / leak
```

3.5 Exception mejtexy

```
void f() {  
    int* p = new int(5);  
    mayThrow(); // ete throw aner -> delete chi hasni -> LEAK  
    delete p;  
}
```

```
g++ -fsanitize=address -g leak.cpp -o leak
./leak
# -> "detected memory leaks", stack trace ov e new-y
```

6. Interview harcer & patasxanner

Memory leak inch e?

Heap memory vor vercvac e bayc chi azatvel, u aylevs hascee chka; mnum e zbaxvac minchev program-i verjy.

Erb e linum?

delete mor'acum, early return/exception, pointer overwrite, delete vs delete[] sxal.

Inchpes xusapel?

RAII + smart pointers (unique_ptr/shared_ptr), STL container-ner (vector/string), amen new-i hamar delete.

Inchpes haytnaberel?

Valgrind, AddressSanitizer/LeakSanitizer, CRT debug heap.

delete vs delete[]?

new -> delete; new[] -> delete[]. Sxal zuygy -> undefined behavior / leak.

Modern C++-um inchpes es xusapum leak-ic?

make_unique/make_shared, ochmi raw new/delete; container-ner. Rule of Zero.

Cheat Sheet

MEMORY LEAK = heap memory vercvac, chi azatvel, hascee korel

PATCHARNER:

delete moracum | early return/exception | pointer overwrite | delete vs delete[]

XUSAPEL:

RAII + smart pointers (make_unique/make_shared)

STL containers (vector/string)

amen new -> delete (+ nullptr)

TOOLS:

valgrind --leak-check=full | -fsanitize=address (ASan) | -fsanitize=leak

RULE: modern C++ -> NO raw new/delete -> no leaks (Rule of Zero)